

# WEATHERGPT

## AI-Powered Conversational Weather Intelligence for India

SIH 2026 — Project Documentation

**TAGLINE** Understand the Weather. Know What to Do.

| Project Information | Details                                                                         |
|---------------------|---------------------------------------------------------------------------------|
| Project Name        | WeatherGPT                                                                      |
| Category            | AI / Weather Intelligence / GIS / Multilingual Technology                       |
| Target Platform     | Web + Android                                                                   |
| Primary Audience    | Citizens, farmers, travellers, researchers and disaster-management stakeholders |
| Competition         | Smart India Hackathon (SIH) 2026                                                |
| Team / Institution  | [ADD TEAM NAME] / [ADD INSTITUTION]                                             |
| Problem Statement   | [ADD OFFICIAL SIH PROBLEM STATEMENT NUMBER AND TITLE]                           |

*Prepared for final project submission*

# 1. Executive Summary

---

*Turning complex weather information into understandable, localized and actionable intelligence.*

WeatherGPT is an AI-powered conversational weather intelligence platform designed to make weather information easier to access, understand and act upon. Instead of requiring users to navigate multiple weather portals, forecast pages, warning bulletins and technical datasets, the platform brings weather information into a unified conversational experience.

The system is designed around a simple principle: weather data is valuable only when people can understand what it means for them. WeatherGPT therefore combines weather observations and forecasts with conversational AI, geospatial information, multilingual interaction and personalized advisories.

## Core Objective

The objective is to build a single platform through which a user can ask natural-language questions about current weather, forecasts, rainfall, travel conditions, alerts, locations and other weather-related topics, while receiving responses grounded in available meteorological data.

## Key Capabilities

- Conversational weather queries using natural language.
- Current weather and forecast presentation.
- Location-aware and PIN-code-based weather lookup.
- Interactive geographic weather visualization.
- Weather warning and alert presentation.
- Indian-language interaction through the project's BHASHINI integration.
- Voice-oriented interaction and read-aloud capabilities where supported.
- Climate and historical weather visualization where data is available.
- Personalization, accessibility and configurable user preferences.

## Project Value Proposition

**DATA → INTELLIGENCE → ACTION** WeatherGPT is positioned as an intelligence layer between complex meteorological information and the everyday decisions users need to make.

## Submission Positioning

For SIH 2026, the project is presented not simply as a weather application, but as a conversational interface over weather intelligence. The platform emphasizes accessibility, Indian-language usability, location awareness, explainable data presentation and actionable guidance.

## 2. Problem Statement & Motivation

*Why weather information needs an intelligence layer.*

### 2.1 The Problem

Weather information is commonly distributed across multiple portals, bulletins, satellite products, forecast systems, maps and technical datasets. For many users, the challenge is not the absence of information; it is the difficulty of finding the relevant information, understanding it and translating it into a practical decision.

| Challenge                  | Impact                                                                             |
|----------------------------|------------------------------------------------------------------------------------|
| Fragmented information     | Users move between multiple services to understand one situation.                  |
| Technical presentation     | Meteorological terminology can be difficult for non-specialist users.              |
| Language barriers          | Important information may not be accessible in a user's preferred Indian language. |
| Location complexity        | A generic city forecast may not answer a user's precise local question.            |
| Warnings vs. understanding | A warning label alone may not explain what action a user should take.              |
| Information overload       | Users can receive many measurements without knowing which ones matter.             |
| Accessibility gaps         | Small text, visual-only indicators or complex interfaces can exclude users.        |

### 2.2 Motivation

A conversational interface can reduce the cognitive effort required to access weather intelligence. Instead of forcing a user to know the correct meteorological term or navigate several screens, the user can ask a question in ordinary language.

**Example** A user can ask “Will I need an umbrella tomorrow evening?” rather than manually interpreting precipitation probability, rainfall amount and forecast timing.

### 2.3 Intended Users

- General citizens planning daily activities.
- Commuters and travellers making short-term decisions.
- Farmers and agricultural users seeking weather-aware guidance.
- Researchers and students exploring weather information.
- Communities monitoring local hazards.
- Potential institutional and disaster-management users requiring clearer access to weather intelligence.

### 2.4 Design Principle

The platform follows the principle that users should not have to become meteorologists to use weather information effectively.

# 3. Proposed Solution

*A unified conversational and visual weather intelligence platform.*

## 3.1 Solution Overview

WeatherGPT combines a web interface and Android application with a backend service layer. The frontend provides the user experience, while the backend coordinates weather data, forecast information, AI processing, location services, language services and other integrations used by the application.

## 3.2 User Experience Model

| User Need                   | WeatherGPT Response                                                                       |
|-----------------------------|-------------------------------------------------------------------------------------------|
| What is the weather now?    | Current conditions for the selected or detected location.                                 |
| What will happen later?     | Hourly and daily forecast information.                                                    |
| Should I carry an umbrella? | A data-grounded conversational advisory using relevant precipitation information.         |
| Is there a warning?         | Location-aware warning and alert information.                                             |
| What about another city?    | Location search and comparative weather information where supported.                      |
| Can I use my language?      | Localized interface and language-processing capabilities supported by the implementation. |
| Can I hear the response?    | Read-aloud / voice interaction where supported.                                           |

## 3.3 Product Modules

- Home / Weather Dashboard — current conditions, forecast highlights and personalized information.
- WeatherGPT — conversational weather intelligence.
- Forecast — hourly and multi-day weather information.
- Map / GIS — geographic weather visualization and available layers.
- Alerts — weather warnings, severity, location and recommended actions.
- Climate — historical and trend-oriented information where available.
- Locations — saved places and location/PIN-based weather.
- Settings — language, theme, accessibility, notifications and related preferences.

## 3.4 What Makes the Approach Different

The core differentiator is the combination of conversational access, location-aware weather data, multilingual interaction, visual GIS context and actionable explanation. The application is intended to answer not only “what is the weather?” but also “what does this mean for me?”

**Design Thesis** WeatherGPT transforms meteorological information into understandable decisions through conversation, visualization and context.

# 4. System Architecture

*Technical structure connecting the interfaces, intelligence layer and external services.*

**PLACEHOLDER** — INSERT YOUR FINAL SYSTEM ARCHITECTURE DIAGRAM HERE.

## 4.1 High-Level Architecture

The platform follows a layered architecture. Client applications communicate with a backend API/service layer. The backend coordinates the data and intelligence services required to answer user requests and populate the application's visual interfaces.

| Layer                      | Responsibilities                                                                                         |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| Presentation Layer         | Web and Flutter interfaces, navigation, responsive layouts, accessibility and user interaction.          |
| Application / API Layer    | Request handling, validation, authentication and orchestration of application operations.                |
| Weather Intelligence Layer | Weather observations, forecasts, warnings and related meteorological processing.                         |
| AI Layer                   | Natural-language understanding, contextual response generation and data-grounded advisory presentation.  |
| Language Layer             | Language processing / translation through the project's BHASHINI integration.                            |
| Geospatial Layer           | Location resolution, maps, geographic visualization and location-specific information.                   |
| Persistence Layer          | User preferences, saved locations, application data and other persistent information used by the system. |

## 4.2 Major Data Flow

1. The user submits a query or interacts with a weather feature.
2. The frontend sends the required request to the backend service.
3. The backend validates the request and determines which service or data source is required.
4. Weather, location, alert, language or other required information is retrieved.
5. Where applicable, relevant weather context is passed to the AI layer.
6. The response is structured for presentation in the web or Android interface.
7. The frontend displays the result with source/timestamp information where available.

## 4.3 Architecture Diagram Notes

When adding the final diagram, show the actual technologies and services used by the current implementation. Do not add technologies merely because they are common in similar systems. The final architecture should exactly match the deployed project.

# 5. AI & Conversational Weather Intelligence

*From a natural-language question to a data-grounded response.*

**PLACEHOLDER** — INSERT YOUR AI QUERY PIPELINE DIAGRAM HERE.

## 5.1 Conversational Workflow

| Stage                       | Purpose                                                                                                        |
|-----------------------------|----------------------------------------------------------------------------------------------------------------|
| User Query                  | Accept a natural-language weather question.                                                                    |
| Language / Input Handling   | Identify the language or normalize the request as supported by the implementation.                             |
| Intent Understanding        | Determine whether the user is asking about current weather, forecast, rain, travel, comparison, advisory, etc. |
| Entity / Context Extraction | Identify location, time period and other relevant context.                                                     |
| Weather Retrieval           | Retrieve the relevant weather or forecast data.                                                                |
| Context Construction        | Prepare data needed for a grounded answer.                                                                     |
| AI Response                 | Generate a concise natural-language explanation/advisory.                                                      |
| Presentation                | Show structured values, source/timestamp and available actions.                                                |

## 5.2 Example Interaction

**USER** “Will it rain tomorrow evening in my location?”

The system identifies the forecast intent, determines the relevant location and time period, retrieves available forecast data and generates an answer based on those data.

**RESPONSE** “Rain is expected during the evening period. Carry rain protection if you are travelling during that window.”

The exact wording and values are generated from the actual implementation and available weather data. The system should not present fabricated values or pretend that an unavailable data source was consulted.

## 5.3 Explainability

WeatherGPT can present a “Why?” or data-explanation view where supported. This should expose the relevant data inputs behind an advisory — such as precipitation probability, rainfall amount, forecast window, source and update time — without exposing private model chain-of-thought.

## 5.4 Personalization

Where user preferences and profile information are available, the interface can use context such as saved locations, preferred language and accessibility settings to make the experience more relevant.

# 6. Multilingual, Voice & Accessibility

*Designing weather intelligence for a linguistically diverse population.*

## 6.1 Indian-Language Support

WeatherGPT is designed to support multilingual interaction using the project's BHASHINI integration. The language experience should cover the languages actually configured and tested in the implementation rather than claiming support for languages that are not operational.

| Capability             | Implementation Goal                                                                   |
|------------------------|---------------------------------------------------------------------------------------|
| Interface localization | Navigation, settings, labels and system messages adapt to the selected language.      |
| Weather responses      | Weather information can be presented in supported languages.                          |
| Translation            | WeatherGPT responses can be translated through the configured language service.       |
| Voice                  | Speech input/output is available where the selected device and service support it.    |
| Long-script handling   | Layouts adapt to longer words, different glyph metrics and script-specific rendering. |
| RTL readiness          | Right-to-left behavior is handled where applicable, particularly for Urdu.            |

## 6.2 Voice Interaction

The voice experience is intended to provide a natural way to interact with weather information. A production interaction should clearly distinguish Idle, Listening, Processing, Speaking and Error states.

## 6.3 Accessibility

- Adjustable text size.
- High-contrast presentation.
- Reduced-motion preference.
- Read-aloud support where available.
- Semantic labels for interactive elements.
- Keyboard accessibility for the web interface where applicable.
- Severity indicators that do not rely on color alone.

## 6.4 Why Accessibility Matters

Weather and emergency information should remain understandable under stressful conditions and across different user abilities. Accessibility is therefore treated as part of the product's information architecture rather than as an optional visual setting.

# 7. GIS, Location & Weather Visualization

*Connecting weather intelligence to geography.*

**PLACEHOLDER** — INSERT YOUR FINAL GIS / MAP DIAGRAM OR DESIGN HERE.

## 7.1 Location Resolution

WeatherGPT can use the location mechanisms implemented in the project, such as device/browser location, manual search and PIN-code lookup.

| Input           | Flow                                                                   |
|-----------------|------------------------------------------------------------------------|
| Device Location | Permission → coordinates → location/weather lookup.                    |
| PIN Code        | PIN validation → location resolution → coordinates → weather/forecast. |
| Manual Search   | Search → selected location → weather and forecast.                     |
| Saved Location  | Stored location → weather/forecast/alert retrieval.                    |

## 7.2 GIS Experience

The GIS interface is intended to provide an interactive geographic view of weather information rather than a static illustration. Where supported by the backend and data providers, the map can expose weather layers, warning overlays, location markers and other meteorological information.

- Map pan and zoom.
- Current-location visualization.
- Location search and PIN-based navigation.
- Weather-layer controls where actual data is available.
- Warning overlays where supported.
- Map legend and data timestamp.

## 7.3 Data Transparency

Weather visualization should distinguish between observations, forecasts, warnings and AI-generated explanations. Where the source and update time are available, they should be visible to the user.

**Trust Principle** A visualization should never imply that a live layer exists when the underlying data source is unavailable.

# 8. Alerts, Personalization & User Experience

---

*Turning weather information into action.*

## 8.1 Alert Center

The alert interface is designed to prioritize understanding and action. A warning should communicate what is happening, where it is happening, when it is valid, how severe it is and what the user should consider doing.

| Alert Element    | Purpose                                                                                |
|------------------|----------------------------------------------------------------------------------------|
| Severity         | Communicate relative urgency using label, iconography and accessible visual treatment. |
| Event            | Identify the weather hazard.                                                           |
| Location         | Show the affected area.                                                                |
| Validity         | Communicate the relevant time window.                                                  |
| Issued / Updated | Show information freshness when available.                                             |
| Source           | Identify the originating authority/service where available.                            |
| Action           | Provide understandable safety-oriented guidance where available.                       |

## 8.2 Personalization

- Preferred language.
- Preferred theme.
- Accessibility preferences.
- Saved locations.
- Location-aware weather experience.
- Notification preferences where supported.

## 8.3 Notification Strategy

Notifications should be used primarily for meaningful weather events rather than creating unnecessary noise. Where the implementation supports notification categories, users should be able to control relevant categories and permissions.

## 8.4 User Experience Philosophy

The interface should answer the most important question first and expose deeper information progressively. A user who only wants the current temperature should not have to understand a complex dashboard, while a more advanced user should be able to explore forecasts, maps, alerts and climate information.

## 8.5 Example Decision Flow

Weather condition → forecast context → risk/alert context → personalized explanation → recommended action.

# 9. Technology Stack, Security & Deployment

*Productionizing the WeatherGPT platform.*

## 9.1 Technology Stack

Replace the placeholders below with the exact technologies used in the final repository.

| Area             | Technology / Service                       |
|------------------|--------------------------------------------|
| Android Frontend | [Flutter version / packages]               |
| Web Frontend     | [Reflex or final web stack]                |
| Backend          | [Python framework / version]               |
| Database         | [Actual database]                          |
| Weather Data     | [Actual provider(s)]                       |
| AI               | [Actual model/provider]                    |
| Language         | BHASHINI / ULCA integration as implemented |
| Maps / GIS       | [Actual map/GIS provider and packages]     |
| Notifications    | [Actual notification technology]           |
| Hosting          | [Actual deployment provider(s)]            |
| Source Control   | Git + GitHub                               |

## 9.2 Security Principles

- Provider API keys and secrets are kept out of client-side source code.
- Production secrets are supplied through environment/configuration mechanisms.
- User input is validated before processing.
- Sensitive authentication/session information is not intentionally exposed in logs.
- Production traffic should use HTTPS.
- CORS and API access policies should be configured for the deployed frontend.
- Error responses should avoid leaking internal implementation details.

## 9.3 Deployment Architecture

**PLACEHOLDER** — INSERT YOUR FINAL DEPLOYMENT ARCHITECTURE DIAGRAM HERE.

The production environment should separate client applications from protected backend credentials. The web application and Flutter application communicate with the public API layer, while provider credentials remain server-side.

## 9.4 Production Environment

- Production backend URL: [ADD URL]
- Production web URL: [ADD URL]
- API documentation URL: [ADD URL]
- GitHub repository: [ADD URL]
- Release APK: [ADD GITHUB RELEASE / ARTIFACT URL]

# 10. Testing, Validation & Performance

*Evidence that the product works beyond the prototype stage.*

## 10.1 Functional Testing Matrix

| Area            | Test                                                              | Status          |
|-----------------|-------------------------------------------------------------------|-----------------|
| Weather         | Current weather loads from production API                         | [PASS / VERIFY] |
| Forecast        | Hourly/daily forecast loads correctly                             | [PASS / VERIFY] |
| WeatherGPT      | Different questions produce contextual responses                  | [PASS / VERIFY] |
| Location        | GPS/manual location flow works                                    | [PASS / VERIFY] |
| PIN             | Valid/invalid PIN handling works                                  | [PASS / VERIFY] |
| Map             | Map renders and interactions work                                 | [PASS / VERIFY] |
| Alerts          | Warnings and severity display correctly                           | [PASS / VERIFY] |
| Language        | Selected language changes relevant UI                             | [PASS / VERIFY] |
| BHASHINI        | Translation/language service works                                | [PASS / VERIFY] |
| Voice           | Microphone/read-aloud flow works where supported                  | [PASS / VERIFY] |
| Notifications   | Permission/settings/delivery path works where supported           | [PASS / VERIFY] |
| Accessibility   | Text size/high contrast/reduced motion work                       | [PASS / VERIFY] |
| Theme           | Light/dark/system theme works and persists                        | [PASS / VERIFY] |
| Offline/Failure | Graceful failure and cached-data behavior works where implemented | [PASS / VERIFY] |

## 10.2 Responsive Testing

Recommended final QA dimensions:

- Web: 360, 390, 430, 600, 768, 1024, 1280, 1440 and 1920 px widths.
- Android: small, standard and large phone displays.
- Portrait and landscape where applicable.

## 10.3 Multilingual QA

Test every language that the final build claims to support. Pay particular attention to long labels, warnings, navigation, text wrapping, script rendering and right-to-left behavior where applicable.

## 10.4 Performance

- Avoid unnecessary API requests and repeated UI rebuilds.
- Use lazy loading for long lists and heavy data.
- Optimize map rendering and assets.
- Dispose Flutter controllers/resources correctly.
- Use appropriate caching while clearly identifying stale data.

## 10.5 Failure Handling

The application should provide useful states for unavailable weather data, failed location permission, invalid PIN, unavailable AI, translation failure, network loss and empty datasets. A graceful failure is preferable to displaying invented or misleading information.

# 11. Implementation Status, Roadmap & Submission Assets

*The final transition from working project to defensible SIH submission.*

## 11.1 Current Implementation

Phase 1 and Phase 2 established the project's backend capabilities and frontend experience. Phase 3 focuses on productionization, deployment, release engineering, documentation and demonstration.

| Phase   | Outcome                                                                                                                              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------|
| Phase 1 | Core backend/services and initial project functionality.                                                                             |
| Phase 2 | Frontend productization across web and Android, with responsive UI, interaction, localization/accessibility and feature integration. |
| Phase 3 | Production deployment, GitHub publication, release APK, documentation, presentation and demonstration.                               |

## 11.2 Recommended Future Roadmap

- Broader meteorological and satellite-data integration.
- Advanced numerical weather prediction model comparison.
- More sophisticated agricultural decision support.
- Disaster-management and institutional dashboards.
- IoT / local weather-station integration.
- More advanced risk and anomaly detection.
- Expanded personalization and proactive weather intelligence.

## 11.3 Final Submission Asset Checklist

| Asset                 | Target Output                                                   |
|-----------------------|-----------------------------------------------------------------|
| Source Code           | GitHub repository with clean README and configuration examples. |
| Backend               | Live HTTPS API with production environment configuration.       |
| Web                   | Live production web application.                                |
| Android               | Signed/release APK tested on a physical device.                 |
| Documentation         | WeatherGPT_Project_Documentation.docx / PDF export.             |
| Presentation          | 5-slide SIH presentation.                                       |
| Demo Video            | Short, polished product demonstration.                          |
| Architecture Diagrams | System, AI/data flow, GIS and deployment diagrams.              |
| Screenshots           | Final web and mobile product screenshots.                       |

## 11.4 Final Links

GitHub: [<https://github.com/Soumen044/weathergpt>]

Live Web: [<https://weathergpt-coral-psi.vercel.app>]

APK Release: [<https://github.com/Soumen044/weathergpt/releases/tag/v1.0.0>]

YouTube Demo: [<https://youtu.be/XvyzF2WNxIU>]

## 11.5 Closing Statement

**WEATHERGPT** Understand the Weather. Know What to Do.

WeatherGPT aims to bridge the gap between complex meteorological information and everyday decisions by combining data, AI, language accessibility, geospatial context and actionable communication in one platform.